

FULL-STACK MODE

Product and implementation plan

A Laravel-like application starter for building complete TypeScript products with Authenik8 already connected from browser to database.

The screenshot displays the Authenik8 dashboard for a user named Alex Morgan. The interface is divided into several sections:

- Header:** Greeting "Good evening, Alex" and a "New project" button.
- Overview Cards:** Three cards showing "Total projects: 12", "Active sessions: 3", and "Security status: Protected".
- Recent projects:** A table listing recent projects with columns for Project, Status, Members, and Updated.
- Active sessions:** A table listing active sessions with columns for Session, Location, IP address, Last active, and Action.
- Security overview:** A section showing the status of various security features like Email & password, Google, GitHub, and Refresh protection.
- API activity:** A section showing the number of requests (1,284) and a line graph of activity over the last 24 hours.

Recent projects table:

Project	Status	Members	Updated
Customer Portal	Production	5	2h ago
Analytics API	Staging	3	5h ago
Mobile Backend	Development	4	1d ago

Active sessions table:

Session	Location	IP address	Last active	Action
Chrome on Linux	San Francisco, US	203.0.113.42	2m ago	Revoke
Mobile app	San Francisco, US	203.0.113.88	15m ago	Revoke

Security overview table:

Feature	Status
Email & password	Enabled
Google	Connected
GitHub	Connected
Refresh protection	Active

API activity: Requests (24h) 1,284, +12.5% vs yesterday.

01 - PRODUCT DIRECTION

Build an application starter, not another backend template

Authenik8 already generates secure Express authentication foundations. Full-stack mode should wrap that engine in a finished application shell: connected frontend pages, secure token handling, shared validation, a dashboard, administration, and one real product module. A developer should finish the wizard and immediately begin writing business features.

Recommended positioning

Market it as the Authenik8 Full-stack Starter or Authenik8 App Kit. Avoid calling it a framework at first. The preset can grow into a broader application platform after its conventions and generators prove useful.

What success looks like

One command

A single CLI flow creates the API, web application, database schema, Redis configuration, environment files, and development scripts.

Secure defaults

No tokens in localStorage. Refresh rotation and server-side session control remain core Authenik8 advantages.

Complete auth UX

Login, registration, OAuth, account security, active sessions, and admin authorization are already connected.

A clear pattern

An example Project resource demonstrates how schemas, services, API calls, policies, hooks, and pages fit together.

Initial preset family

Preset	Purpose	Status
express-base	Minimal Express and TypeScript starting point	Existing
express-auth	Credentials-based authenticated API	Existing
express-auth+	Credentials, OAuth, RBAC, Redis-backed sessions	Existing
fullstack	Finished React application plus the Authenik8 API	New flagship

The design reference is Laravel's starter-kit philosophy: backend authentication is useful, but the real acceleration comes from pairing it with finished routes, views, and application conventions.

02 - GENERATION EXPERIENCE

The CLI should feel like a product setup wizard

The first release should remain opinionated. Too many framework and styling questions will make the preset harder to maintain and reduce the chance that every generated combination is tested.

```
$ npx create-authenik8-app my-saas

? What would you like to build?
  Express API
  Express API + OAuth
> Full-stack application (recommended)

? Authentication methods
> Email and password
> Google
> GitHub

? Include application modules?
> User dashboard
> Profile and security settings
> Admin area
> Example project CRUD

? Database
> PostgreSQL
  SQLite for development

Creating my-saas...

OK Express API configured
OK React frontend created
OK Authenik8 Identity Engine connected
OK Database schema and Redis configured
OK Authentication UI connected
OK Admin authorization configured

$ cd my-saas && npm install
$ docker compose up -d
$ npm run dev
```

First-release rule

Offer one golden path: React + Vite + Express + Prisma + PostgreSQL + Redis. Add Next.js, Vue, alternate ORMs, and deployment providers only after the main preset is stable.

Generation guarantees

- The generated project installs and starts with documented commands.
- The database migration and seed complete without manual file edits.
- Credential login, OAuth callbacks, refresh, logout, and protected routes work end to end.
- A generated administrator can access the admin area while normal users cannot.
- The example resource works from database to browser and includes tests.
- The README explains local development, production requirements, environment variables, and the threat model.

03 - TECHNICAL ARCHITECTURE

One repository, one origin, clear application boundaries

For the first full-stack preset, Vite should proxy /api requests to Express in development. In production, Express can serve the compiled frontend. This keeps the browser and API on one origin, which makes secure refresh cookies and deployment easier than a split-domain setup.

apps/web React, Vite, React Router, TanStack Query, Tailwind, reusable accessible components, protected layouts, and finished product pages.	apps/api Express, Authenik8 core, Prisma, Redis, controllers, services, policies, OpenAPI documentation, health checks, and tests.
packages/contracts Shared Zod schemas and TypeScript types for auth, users, projects, pagination, and error responses.	packages/api-client One generated client that attaches the in-memory access token, normalizes errors, refreshes once, and retries safely.

Request and session flow

Step	Browser	API and Authenik8
1	Submits login or completes OAuth	Resolves the identity and validates credentials/provider profile
2	Receives a short-lived access token	Stores the rotating refresh session in Redis
3	Keeps access token in memory	Sets refresh token in a restricted HttpOnly cookie
4	Sends Bearer token to protected API routes	Verifies access token and enforces role/policy
5	Refreshes once when access expires	Atomically rotates refresh token and rejects replay
6	Logout clears local auth state	Revokes Redis session and clears refresh cookie

Important boundary

Frontend route guards improve user experience; they are never the security boundary. Every sensitive API route must still enforce authentication, roles, ownership, or an explicit policy on the server.

04 - GENERATED PROJECT

The generated code must be understandable without Authenik8 knowledge

A workspace layout makes the deployment units obvious and provides a clean home for shared contracts. It also allows a later mobile app, worker, or documentation site to join the repository without reorganizing the entire project.

```
my-saas/
|-- apps/
|   |-- api/
|   |   |-- prisma/schema.prisma
|   |   |-- src/
|   |       |-- auth/
|   |       |-- modules/{users,projects}/
|   |       |-- middleware/
|   |       |-- config/
|   |       |-- app.ts, server.ts
|   |-- web/
|   |   |-- src/
|   |       |-- pages/{auth,dashboard,settings,admin}/
|   |       |-- components/
|   |       |-- auth/
|   |       |-- hooks/
|   |       |-- api/
|-- packages/{contracts,api-client,ui}/
|-- docker-compose.yml
|-- package.json
|-- README.md
```

Module convention

Layer	Responsibility	Project example
Contract	Input validation and public response types	project.schema.ts
Repository	Prisma queries only	project.repository.ts
Service	Business rules and transactions	project.service.ts
Controller	HTTP request and response mapping	project.controller.ts
Policy	Role, ownership, or permission decision	project.policy.ts
Route	Middleware order and endpoint declaration	project.routes.ts
Web feature	Queries, forms, pages, and UI state	features/projects/

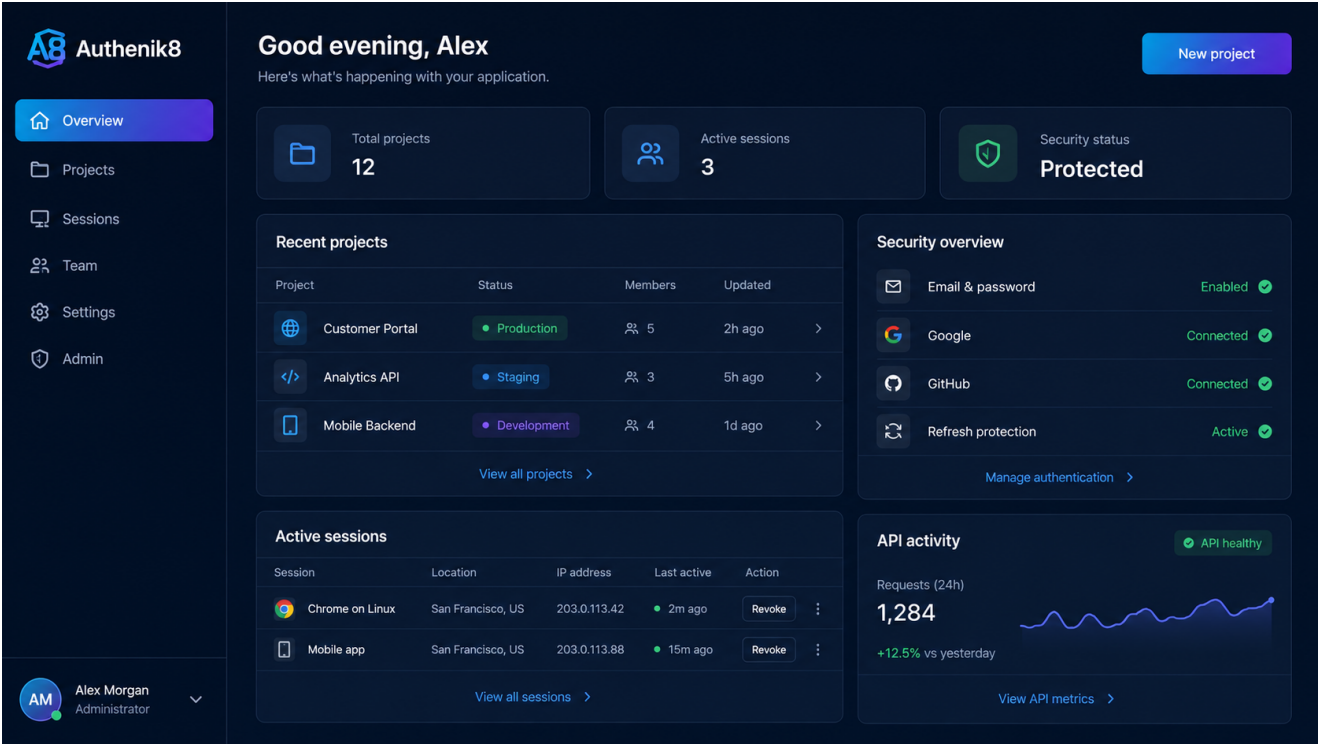
Why include a Project module?

Authentication pages prove that login works. A real CRUD module proves that the entire application architecture works and gives developers a pattern they can copy with confidence.

05 - VISUAL APPLICATION EXPERIENCE

A calm, security-focused SaaS shell

The generated interface should look useful without dictating the user's final product. Use a restrained dark navy theme, cyan-to-violet accents, clear status colors, generous spacing, compact data tables, and accessible contrast. The mockup below represents the default dashboard direction.



Area	Included screens and states
Authentication	Login, registration, OAuth buttons, error states, loading states
Recovery	Forgot password, reset password, expired or invalid token
Verification	Verify-email notice, resend action, successful verification
Dashboard	Welcome summary, projects, sessions, security status, API health
Settings	Profile, password, linked providers, active sessions
Administration	Users, roles, status, session revocation, audit trail
Example resource	Project list, create, details, edit, delete, empty and error states

The default UI should be easy to remove or restyle. Components and page shells belong in packages/ui and apps/web, while authentication behavior belongs in the auth provider and generated API client.

06 - AUTHENTICATION AND SECURITY

Preserve Authenik8's strongest product difference

Full-stack mode should make the secure path the easiest path. The UI must not weaken the Redis-backed token lifecycle that already differentiates Authenik8 from ordinary JWT boilerplate.

Access token

Short lived and held in application memory. Sent as an Authorization Bearer token to protected API endpoints.

Rotation

Each successful refresh replaces the previous token atomically. Reuse and concurrent refresh attempts fail.

Authorization

RBAC protects broad roles; policies handle object ownership and business-specific permissions.

Refresh token

Stored in an HttpOnly, Secure, SameSite cookie scoped as narrowly as practical to the refresh endpoint.

Session control

Redis remains the source of truth for active refresh sessions, logout, revocation, and replay protection.

OAuth

State is validated, provider profiles are normalized, verified-email requirements remain enforced, and linking is explicit.

Required production defenses

- Exact origin allowlist and an Origin check on cookie-driven authentication endpoints.
- HTTPS, Secure cookies, trusted proxy configuration, Helmet, and Redis-backed rate limiting.
- Generic credential errors that do not reveal whether an email address exists.
- Strong password hashing, email verification, reset-token expiry, and single-use recovery tokens.
- Server-side ownership checks for every resource, not only role checks.
- Redacted structured logs, request IDs, health checks, and a reviewed threat model.
- Integration tests covering login, refresh rotation, replay, logout, OAuth state, authorization, and session revocation.

Browser storage rule

The generated application must never place access tokens or refresh tokens in localStorage. This should be documented and covered by a template-level test or lint rule where practical.

07 - LARAVEL-LIKE DEVELOPER EXPERIENCE

The preset starts the app; generators make it feel like a platform

create-authenik8-app should remain responsible for creating a project. A later project CLI can provide commands inside generated applications. Separating these responsibilities keeps the creator focused and lets the project CLI evolve independently.

```
$ npx authenik8 make:resource Project \
  --fields "name:string,description:text,status:enum"

Generated:
  Prisma model and migration
  Zod request and response contracts
  Repository, service, controller, policy, routes
  React Query hooks and typed API methods
  List, create, edit, and details pages
  API integration tests

$ npx authenik8 make:policy Project
$ npx authenik8 make:admin-resource User
$ npx authenik8 auth:add google
$ npx authenik8 doctor
```

Recommended command sequence

Command	Value	When to build
authenik8 doctor	Checks Node, database, Redis, secrets, migrations, URLs, and common deployment mistakes	First
make:resource	Generates one complete vertical feature slice	After preset stabilizes
make:policy	Creates a typed authorization decision and tests	With resource generator
make:admin-resource	Generates admin tables and actions	Later
auth:add	Adds provider config, environment fields, routes, and UI	Later

Generator safety

Resource generation must be deterministic, idempotent where possible, and protected by a dry-run or preview mode. Editing Prisma schemas and route indexes requires parsing or structured templates, not fragile string replacement.

08 - DELIVERY ROADMAP

Ship the smallest version that proves the complete promise

Phase	Deliverables	Exit condition
1. Foundation	Workspace, React/Vite app, Express API, shared contracts, Docker services, unified dev command	Fresh generation starts cleanly
2. Auth UX	Register, login, refresh, logout, OAuth, protected layout, auth errors	Auth works end to end
3. Account	Profile, password, linked providers, active sessions, revocation	Users manage their identity
4. Admin	User list, roles, status, admin route and API protection	Admin boundaries are tested
5. Product proof	Project CRUD, ownership policy, hooks, pages, tests	One complete feature slice works
6. Hardening	CI, threat model, rate-limit tests, security headers, production guide	Release checklist passes
7. Tooling	doctor command, then resource and policy generators	Generated code is deterministic

Deliberately postpone

- Next.js, Vue, Svelte, Fastify, and multiple visual themes.
- Billing, subscriptions, multi-tenancy, teams, invitations, and enterprise SSO.
- A plugin marketplace or a general-purpose framework runtime.
- Automatic cloud provisioning across many deployment platforms.

Release principle

Every option multiplies the combinations that need installation, migration, build, browser, API, and security testing. The flagship preset should be excellent before the matrix becomes wide.

MVP acceptance checklist

- A new user can generate, install, migrate, seed, and run the application from the README.
- Registration, login, OAuth, protected routes, refresh, replay rejection, and logout are verified by automated tests.
- The UI includes responsive loading, empty, validation, forbidden, and failure states.
- A normal user cannot reach admin APIs even by bypassing the frontend.
- The example Project feature enforces ownership on every read and write.
- Production documentation covers secrets, HTTPS, Redis exposure, cookies, CORS/origins, proxies, email, backups, and observability.

09 - RECOMMENDED DECISIONS

The concrete plan

Preset name

Use the CLI identifier fullstack. Present it publicly as Authenik8 Full-stack Starter or Authenik8 App Kit.

Repository model

Workspace with apps/api, apps/web, packages/contracts, packages/api-client, and packages/ui.

Token model

Access token in memory; rotating refresh token in a restricted HttpOnly cookie; Redis-backed session truth.

CLI strategy

Keep create-authenik8-app for creation. Introduce a separate in-project authenik8 CLI later.

First frontend

React with Vite. Keep Next.js as a later dedicated preset rather than mixing architectures.

Deployment model

One origin by default. Express serves the compiled web application in production.

Proof feature

A Project CRUD module with ownership policy and end-to-end tests.

Scope discipline

Stabilize the golden path before frameworks, billing, teams, deployment providers, or plugins.

Immediate build sequence

Order	Implementation task
1	Write the fullstack preset contract: exact files, scripts, environment variables, ports, and testable guarantees.
2	Create the workspace and make npm run dev start web and API together.
3	Connect credential login, refresh-cookie flow, auth provider, protected layout, and logout.
4	Add OAuth callbacks, linked-account UI, profile, security, and sessions.
5	Add admin authorization and the Project vertical slice.
6	Build the generated-project test harness and run it in the CLI repository's CI.

Final product promise

Run one command and receive a secure, understandable full-stack TypeScript application - not an empty dashboard, not disconnected auth pages, and not a folder of boilerplate the developer must finish wiring.